

BÁO CÁO TIẾN ĐỘ LUẬN VĂN — Phần 2

Critic-only ADP với Concurrent Learning
Loại bỏ yêu cầu Persistence of Excitation

Nguyễn Thành Trung

Chuyên ngành: Kỹ thuật Điều khiển và Tự động hóa — GVHD: PGS.TS. Nguyễn Hoài Nam

Đại học Bách khoa Hà Nội — Tháng 05/2026

Công việc đã hoàn thành:

1. Nghiên cứu lý thuyết Critic-only ADP (giảm từ 2 mạng xuống 1 mạng)
2. Thiết kế Concurrent Learning (CL): dùng history stack thay tín hiệu PE
3. Cài đặt bộ điều khiển CL trên MATLAB, so sánh với AC-ADP (SM1) và Backstepping
4. Mô phỏng trên 2 quỹ đạo (tròn, thẳng), phân tích hội tụ trọng số và sai số Bellman
5. Kết luận: CL đạt hiệu năng tương đương AC+PE mà **không cần PE**

1 Động cơ và bối cảnh

1.1 Nhìn lại kết quả SM1 và các hạn chế

Trong báo cáo Seminar 1 (SM1), chúng tôi đã cài đặt thành công bộ điều khiển Actor-Critic ADP (Vamvoudakis & Lewis, 2010) cho bài toán bám quỹ đạo robot di động bánh xe (WMR). Phương pháp này sử dụng hai mạng nơ-ron: mạng Critic để xấp xỉ hàm giá trị tối ưu, và mạng Actor để xấp xỉ chính sách điều khiển tối ưu. Kết quả cho thấy Actor-Critic ADP hoạt động tốt trên quỹ đạo đường tròn, tuy nhiên bộc lộ nhiều hạn chế khi áp dụng trên quỹ đạo đường thẳng.

Cụ thể, bốn hạn chế chính đã được xác định:

1. **Cần khởi tạo “warm start” (admissible initial policy):** Nếu khởi tạo trọng số ban đầu $W_0 = 0$, hệ thống mất ổn định ngay lập tức. Lý do là khi $W_0 = 0$, chính sách điều khiển ban đầu $u_o = 0$ (không có feedback), hệ thống mở hoạt động như không có bộ điều khiển. Việc chọn W_0 phù hợp (đảm bảo chính sách ban đầu ổn định) là một yêu cầu khó khăn trong thực tế.
2. **Tín hiệu PE (Persistence of Excitation) phụ thuộc quỹ đạo:** PE là tín hiệu nhiễu bên ngoài được cộng vào lệnh điều khiển nhằm “kích thích” hệ thống, giúp các trọng số mạng nơ-ron hội tụ. Trên đường tròn ($\omega_r \neq 0$), bản thân quỹ đạo đã có sự kích thích tự nhiên (vận tốc góc ω_r tạo coupling giữa các trạng thái), nên PE cộng thêm là có lợi. Tuy nhiên, trên đường thẳng ($\omega_r = 0$), PE trở thành nhiễu thuần túy, phá hỏng hiệu năng bám quỹ đạo.

3. **Hai mạng, nhiều siêu tham số:** Với các tham số $\alpha_c, \alpha_{a1}, \alpha_{a2}, \kappa_c$, cùng với biên độ, tần số PE — việc tinh chỉnh (tuning) hoàn toàn bằng tay và rất nhạy cảm với điều kiện vận hành.
4. **Hội tụ chậm trên đường thẳng:** $z_{\text{rms}} = 0.36$ m (trung bình bình phương của sai lệch) sau 120 s vẫn còn rất lớn, cho thấy trọng số chưa hội tụ đến nghiệm tối ưu.

1.2 Mục tiêu của SM2

Từ những hạn chế nêu trên, mục tiêu của Seminar 2 được xác định rõ ràng:

- **Giảm từ 2 mạng xuống 1 mạng** (Critic-only): loại bỏ mạng Actor, suy chính sách trực tiếp từ mạng Critic duy nhất.
- **Loại bỏ hoàn toàn yêu cầu PE** bằng kỹ thuật Concurrent Learning (CL): sử dụng dữ liệu lịch sử thay vì tín hiệu kích thích bên ngoài.

Nếu thành công, bộ điều khiển mới sẽ đơn giản hơn (1 mạng, ít siêu tham số), đồng thời hoạt động tốt trên mọi loại quỹ đạo mà không phụ thuộc vào điều kiện PE.

2 Lý thuyết: Critic-only ADP

2.1 Từ hai mạng sang một mạng

Như đã trình bày trong SM1, kiến trúc Actor-Critic ADP sử dụng hai mạng nơ-ron riêng biệt:

- **Mạng Critic** xấp xỉ hàm giá trị (value function):

$$\hat{V}(z) = \hat{W}_c^T \phi(z) \quad (1)$$

trong đó $\hat{W}_c \in \mathbb{R}^l$ là vector trọng số Critic, $\phi(z) \in \mathbb{R}^l$ là vector hàm cơ sở (basis function). Hàm giá trị $\hat{V}(z)$ đánh giá “chi phí tương lai” của việc ở trạng thái z hiện tại.

- **Mạng Actor** xấp xỉ chính sách điều khiển tối ưu:

$$\hat{u}_o = -\frac{1}{2} R^{-1} g(z)^T \nabla \phi^T \hat{W}_a \quad (2)$$

trong đó $\hat{W}_a \in \mathbb{R}^l$ là vector trọng số Actor, $g(z)$ là ma trận đầu vào của hệ thống, $\nabla \phi = \partial \phi / \partial z$ là Jacobian của hàm cơ sở.

Mục đích của quá trình học là cả hai vector trọng số đều hội tụ về giá trị tối ưu:

$$\hat{W}_a \rightarrow \hat{W}_c \rightarrow W^* \quad (3)$$

Ý nghĩa vật lý: khi quá trình học kết thúc, trọng số Actor và Critic đều xấp xỉ cùng một giá trị W^* — nghiệm của phương trình Hamilton-Jacobi-Bellman (HJB). Một câu hỏi tự nhiên đặt ra: *nếu cuối cùng $\hat{W}_a \approx \hat{W}_c$, tại sao không dùng trực tiếp $\hat{W}_c$ để tính chính sách điều khiển?*

Cách tiếp cận Critic-only: Loại bỏ hoàn toàn mạng Actor, suy chính sách điều khiển trực tiếp từ mạng Critic:

$$u_o = -\frac{1}{2}R^{-1}g(z)^\top \nabla \phi^\top W \quad (4)$$

trong đó $W \in \mathbb{R}^l$ là vector trọng số Critic *duy nhất*. Số tham số cần học giảm từ $2l = 12$ (6 Critic + 6 Actor) xuống còn $l = 6$.

Ưu điểm của Critic-only so với Actor-Critic:

- **Ít tham số học hơn:** Chỉ còn 6 trọng số thay vì 12, giảm độ phức tạp tính toán và tăng tính ổn định của quá trình học.
- **Loại bỏ luật cập nhật Actor:** Không còn các siêu tham số α_{a1}, α_{a2} của Actor update, giảm khó khăn khi tuning.
- **Chính sách thay đổi “mượt”:** Vì u_o được tính trực tiếp từ W , không có độ trễ giữa Critic và Actor như trong kiến trúc hai mạng. Điều này giúp chính sách điều khiển biến đổi liên tục và trơn hơn theo thời gian.

Vấn đề mới phát sinh: Mặc dù đơn giản hơn về cấu trúc, Critic-only vẫn gặp phải vấn đề cơ bản: nếu chỉ sử dụng online gradient descent (cập nhật trọng số dựa trên dữ liệu hiện tại), hệ thống vẫn cần điều kiện PE để đảm bảo hội tụ. Đây chính là lý do cần kết hợp với **Concurrent Learning**.

2.2 Sai số Bellman và cập nhật online

Để hiểu cơ chế học của Critic-only, ta cần định nghĩa sai số Bellman — thước đo “khoảng cách” giữa chính sách hiện tại và chính sách tối ưu.

Trước hết, định nghĩa vector đạo hàm cơ sở theo động học:

$$\sigma = \nabla \phi(f + g u_o) \in \mathbb{R}^6 \quad (5)$$

Ý nghĩa vật lý: σ là tốc độ thay đổi của hàm cơ sở $\phi(z)$ theo thời gian, dọc theo quỹ đạo của hệ thống khi áp dụng chính sách u_o . Nó phản ánh cách hệ thống “di chuyển” trong không gian trạng thái.

Sai số Bellman được định nghĩa:

$$\delta = W^\top \sigma + z^\top Q z + u_o^\top R u_o \quad (6)$$

Ý nghĩa: δ đo lường mức độ vi phạm phương trình HJB. Khi $W = W^*$ (nghiệm tối ưu), $\delta = 0$. Do đó, mục tiêu của quá trình học là điều chỉnh W sao cho $\delta \rightarrow 0$.

Thành phần $z^\top Q z$ đại diện cho chi phí sai lệch trạng thái (càng lớn khi robot đi lệch quỹ đạo), và $u_o^\top R u_o$ đại diện cho chi phí năng lượng điều khiển (càng lớn khi lệnh điều khiển càng mạnh).

Luật cập nhật online (gradient descent):

$$\dot{W}_{\text{online}} = -\alpha \bar{\sigma} \delta, \quad \bar{\sigma} = \frac{\sigma}{(1 + \sigma^\top \sigma)^2} \quad (7)$$

trong đó $\alpha > 0$ là tốc độ học (learning rate), $\bar{\sigma}$ là phiên bản chuẩn hóa của σ (tránh phát tán số khi $\|\sigma\|$ lớn).

Hạn chế của cập nhật online thuần túy: Tại mỗi thời điểm, chỉ có *duy nhất một hướng* $\bar{\sigma}$ được sử dụng để cập nhật W . Trong không gian $\mathbb{R}^6$, cần ít nhất 6 hướng độc lập tuyến tính để xác định đầy đủ W^* . Điều này đòi hỏi PE — tín hiệu bên ngoài “kích thích” σ quay qua nhiều hướng khác nhau. Không có PE, học chậm và có thể không hội tụ.

3 Concurrent Learning — Thay thế PE

3.1 Ý tưởng chính

Concurrent Learning (CL), đề xuất bởi Chowdhary & Johnson (2010), giải quyết vấn đề PE bằng một ý tưởng đơn giản nhưng hiệu quả: *thay vì chỉ dùng dữ liệu hiện tại, hãy lưu lại và tái sử dụng dữ liệu cũ*.

Cụ thể, ta xây dựng một *history stack* (bộ nhớ lịch sử):

$$\mathcal{H} = \{(\sigma_j, c_j)\}_{j=1}^{N_H} \quad (8)$$

trong đó:

- $\sigma_j = \nabla \phi(z_j) (f(z_j) + g(z_j) u_{o,j})$: vector đạo hàm cơ sở tại thời điểm t_j
- $c_j = z_j^\top Q z_j + u_{o,j}^\top R u_{o,j}$: chi phí tức thời tại t_j

Mỗi cặp (σ_j, c_j) được ghi nhận định kỳ (mỗi 50 ms) trong suốt quá trình vận hành. Ý nghĩa vật lý: đây là “kinh nghiệm” của hệ thống — mỗi điểm dữ liệu ghi lại một “khoảnh khắc” của quá trình vận hành, bao gồm cả hướng chuyển động (σ_j) và chi phí (c_j).

Tại mỗi bước cập nhật, ta tính **sai số Bellman cho từng điểm lịch sử** sử dụng trọng số W hiện tại:

$$\delta_j = W^\top \sigma_j + c_j, \quad j = 1, \dots, N_H \quad (9)$$

Ý nghĩa: với trọng số W hiện tại, nếu quay lại thời điểm t_j , sai số Bellman sẽ là bao nhiêu? Đây là cách “hỏi lại kinh nghiệm cũ” với “kiến thức mới”.

Luật cập nhật CL:

$$\dot{W}_{\text{CL}} = -\frac{\alpha_h}{N_H} \sum_{j=1}^{N_H} \bar{\sigma}_j \delta_j \quad (10)$$

trong đó $\alpha_h > 0$ là tốc độ học từ dữ liệu lịch sử, $\bar{\sigma}_j = \sigma_j / (1 + \sigma_j^\top \sigma_j)^2$ là phiên bản chuẩn hóa.

Tại sao CL thay thế được PE? Mỗi điểm lịch sử σ_j được ghi tại các thời điểm khác nhau, khi trạng thái z_j khác nhau, nên các hướng σ_j cũng khác nhau. Tổng $\sum \bar{\sigma}_j \delta_j$ tương đương một “tích phân” trên nhiều hướng trong $\mathbb{R}^6$, cung cấp đủ thông tin để xác định W^* mà *không cần thêm nhiều kích thích bên ngoài*.

3.2 Điều kiện rank (thay thế PE)

Trong lý thuyết Actor-Critic truyền thống, điều kiện PE yêu cầu tín hiệu $\sigma(t)$ phải “quét” qua đủ mọi hướng trong $\mathbb{R}^l$ trong một khoảng thời gian. Đây là điều kiện khó kiểm tra và phụ thuộc vào quỹ đạo tham chiếu.

Với Concurrent Learning, điều kiện PE được thay thế bởi điều kiện rank đơn giản hơn nhiều:

$$\text{rank}(\Sigma) = l, \quad \Sigma = [\sigma_1, \sigma_2, \dots, \sigma_{N_H}] \in \mathbb{R}^{l \times N_H} \quad (11)$$

Ý nghĩa: ma trận Σ (gồm các vector σ_j xếp thành cột) cần có hạng đầy đủ, tức là các điểm dữ liệu trong history stack phải “đủ đa dạng” về hướng.

Với $l = 6$ (số trọng số cần học), chỉ cần $N_H \geq 6$ điểm dữ liệu độc lập tuyến tính. Trong thực tế, với $N_H = 200$ điểm và ghi mỗi 50 ms, điều kiện rank được thỏa mãn dễ dàng chỉ sau vài giây vận hành (khi robot còn đang bám về quỹ đạo, trạng thái thay đổi nhanh, tạo ra các σ_j đa dạng).

So với PE, điều kiện rank có hai ưu điểm lớn:

1. **Dễ kiểm tra:** Chỉ cần tính rank của ma trận Σ (một phép toán đơn giản), thay vì kiểm tra tích phân tín hiệu theo thời gian.
2. **Không phụ thuộc quỹ đạo:** Mọi quỹ đạo (đường tròn, đường thẳng, hay bất kỳ) đều tạo ra dữ liệu đủ đa dạng trong giai đoạn đầu (khi sai lệch còn lớn).

3.3 Tổng hợp cập nhật trọng số

Kết hợp cả ba thành phần, luật cập nhật trọng số đầy đủ là:

$$\dot{W} = \underbrace{-\alpha \bar{\sigma} \delta}_{\text{online}} - \underbrace{\frac{\alpha_h}{N_H} \sum_j \bar{\sigma}_j \delta_j}_{\text{Concurrent Learning}} - \underbrace{\kappa W}_{\sigma\text{-modification}} \quad (12)$$

Ba thành phần và vai trò của chúng:

1. **Online gradient** ($\alpha = 0.5$): Cập nhật tức thời từ dữ liệu hiện tại. Thành phần này phản ứng nhanh với thay đổi của hệ thống, nhưng chỉ cung cấp thông tin theo một hướng $\bar{\sigma}$.
2. **Concurrent Learning gradient** ($\alpha_h = 0.5$): Cập nhật từ dữ liệu lịch sử (“kinh nghiệm”). Thành phần này cung cấp thông tin theo nhiều hướng $\bar{\sigma}_j$ khác nhau, bù đắp cho sự thiếu thông tin của online gradient.

3. **σ -modification** ($\kappa = 0.01$): Thành phần weight decay, kéo W về gần góc tọa độ. Vai trò: chống phát tán trọng số (weight drift) khi dữ liệu nhiều hoặc mô hình không chính xác.

3.4 Cài đặt vectorized trong MATLAB

Trong MATLAB, việc dùng vòng lặp **for** trên $N_H = 200$ điểm là chậm do MATLAB là ngôn ngữ thông dịch. Giải pháp là vectorize — biến đổi các phép tính thành phép toán ma trận:

Gọi $S = [\sigma_1, \dots, \sigma_{N_H}] \in \mathbb{R}^{6 \times N_H}$ và $C = [c_1, \dots, c_{N_H}] \in \mathbb{R}^{1 \times N_H}$, ta có:

$$\Delta = W^T S + C \in \mathbb{R}^{1 \times N_H} \quad (\text{Bellman error cho toàn bộ history}) \quad (13)$$

$$\bar{S} = S \oslash (1 + \|S_{:,j}\|^2)^2 \in \mathbb{R}^{6 \times N_H} \quad (\text{chuẩn hóa theo cột}) \quad (14)$$

$$\dot{W}_{CL} = -\frac{\alpha_h}{N_H} \bar{S} \Delta^T \quad (\text{tích ma trận duy nhất}) \quad (15)$$

trong đó phép chia $\oslash$ thực hiện theo cột (element-wise division). Toàn bộ việc tính CL gradient quy về một phép nhân ma trận $\mathbb{R}^{6 \times N_H} \times \mathbb{R}^{N_H \times 1}$, nhanh hơn vòng **for** khoảng 100× trong MATLAB.

4 Thiết lập mô phỏng

4.1 Cấu hình chung

Mô phỏng được thực hiện trên MATLAB R2023a với các thiết lập sau:

- **Mô hình:** Kinematic-only (chỉ vòng ngoài động học, không xét vòng trong động lực học). Mô hình sai lệch bám quỹ đạo:

$$\dot{z} = f(z) + g(z) (u_r + u_o) \quad (16)$$

với $z = [z_x, z_y, z_\theta]^T$ là sai lệch vị trí/góc, u_r là lệnh feedforward, u_o là lệnh feedback tối ưu.

- **Phương pháp tích phân:** Euler hiện (explicit), bước thời gian $\Delta t = 0.001$ s, tổng thời gian mô phỏng $T = 120$ s (120,000 bước).
- **Sai lệch ban đầu:** $z_0 = q_0 - q_{r,0} = [0.2, -0.1, 0.1]$ (m, m, rad). Ý nghĩa: robot bắt đầu lệch 20 cm theo phương x , lệch -10 cm theo phương y , và lệch 0.1 rad ($\approx 5.7^\circ$) về hướng.
- **Quỹ đạo tham chiếu:**
 - Đường tròn: bán kính $R = 2$ m, vận tốc dài $v_r = 0.3$ m/s, vận tốc góc $\omega_r = v_r/R = 0.15$ rad/s.
 - Đường thẳng: $v_r = 0.3$ m/s, $\omega_r = 0$ (không quay).

Hai quỹ đạo này được chọn vì chúng đại diện cho hai trường hợp cực đoan: đường tròn có kích thích tự nhiên (coupling giữa các trạng thái qua ω_r), còn đường thẳng không có kích thích nào. Phương pháp nào hoạt động tốt trên cả hai quỹ đạo mới thực sự robust.

4.2 Bốn phương pháp so sánh

Để đánh giá hiệu quả của Concurrent Learning, chúng tôi so sánh bốn phương pháp:

1. **Backstepping (BS)**: Phương pháp model-based cổ điển (Kanayama et al., 1990), với hệ số $(k_1, k_2, k_3) = (1.5, 3, 2)$. Đây là baseline — hiệu năng tốt nhất có thể đạt được khi biết chính xác mô hình.
2. **AC + PE**: Actor-Critic ADP với tín hiệu PE (bộ điều khiển từ SM1). PE được bật trong 30 giây đầu, sau đó tắt ($t_{PE} = 30$ s).
3. **AC (không PE)**: Actor-Critic ADP không có PE. Mục đích: chứng minh PE là cần thiết cho Actor-Critic — nếu bỏ PE, Actor-Critic không hội tụ.
4. **CL (không PE)**: Critic-only + Concurrent Learning — phương pháp đề xuất trong SM2 này. **Không sử dụng PE.**

4.3 Tham số bộ điều khiển CL

Bảng 1 liệt kê các tham số của bộ điều khiển Critic-only + Concurrent Learning, cùng với ý nghĩa vật lý của từng tham số.

Table 1: Tham số bộ điều khiển Critic-only + Concurrent Learning

Tham số	Giá trị	Ý nghĩa
α	0.5	Tốc độ học online (từ dữ liệu hiện tại)
α_h	0.5	Tốc độ học CL (từ dữ liệu lịch sử)
κ	0.01	Hệ số σ -modification (chống phát tán)
$N_H^{\max}$	200	Số điểm lịch sử tối đa trong stack
Δt_{record}	0.05 s	Chu kỳ ghi dữ liệu (mỗi 50 ms)
W_0	$[3, 3, 3, 0, 0, 0]^T$	Khởi tạo warm start (tương đương bộ P)
$u_{o,\max}$	1.5	Giới hạn biên độ feedback
Q	$10 I_3$	Ma trận trọng số sai lệch trạng thái
R	$2 I_2$	Ma trận trọng số năng lượng điều khiển

Giải thích về $W_0 = [3, 3, 3, 0, 0, 0]^T$: ba giá trị đầu (tương ứng với các hàm cơ sở bậc hai z_x^2, z_y^2, z_θ^2) tạo ra chính sách ban đầu tương tự bộ điều khiển tỉ lệ (P-controller), đảm bảo hệ thống ổn định ngay từ đầu. Ba giá trị sau (tương ứng với các thành phần chéo $z_x z_y, z_x z_\theta, z_y z_\theta$) được khởi tạo bằng 0 và sẽ được học trong quá trình vận hành.

5 Kết quả mô phỏng

5.1 Bảng tổng hợp kết quả

Bảng 2 tổng hợp kết quả của bốn phương pháp trên hai quỹ đạo, với ba chỉ số đánh giá:

- $J_c = \int_0^T (z^\top Q z + u_o^\top R u_o) dt$: tổng chi phí tối ưu (càng thấp càng tốt)
- $J_e = \int_0^T \|z\|^2 dt$: sai lệch tích lũy (chỉ tính sai lệch, không tính năng lượng)
- z_{rms} (10 s cuối): sai lệch trung bình bình phương trong 10 giây cuối, đánh giá hiệu năng xác lập

Table 2: So sánh bốn phương pháp trên hai quỹ đạo (kinematic-only, $T = 120$ s)

Quỹ đạo	Phương pháp	J_c	J_e	z_{rms} (10 s cuối)
Đường tròn	BS	1.35	0.12	≈ 0
Đường tròn	AC + PE	7.64	0.73	0.0003
Đường tròn	AC (không PE)	12.48	1.21	0.0013
Đường tròn	CL (không PE)	8.31	0.76	0.0005
Đường thẳng	BS	1.29	0.11	≈ 0
Đường thẳng	AC + PE	72.71	7.25	0.3000
Đường thẳng	AC (không PE)	53.46	5.33	0.2400
Đường thẳng	CL (không PE)	5.60	0.50	0.0042

Nhận xét tổng quát: Backstepping luôn tốt nhất (model-based, không cần học). Trong các phương pháp ADP, CL cho kết quả tốt nhất trên cả hai quỹ đạo, đặc biệt vượt trội trên đường thẳng.

5.2 Phân tích chi tiết trên đường tròn

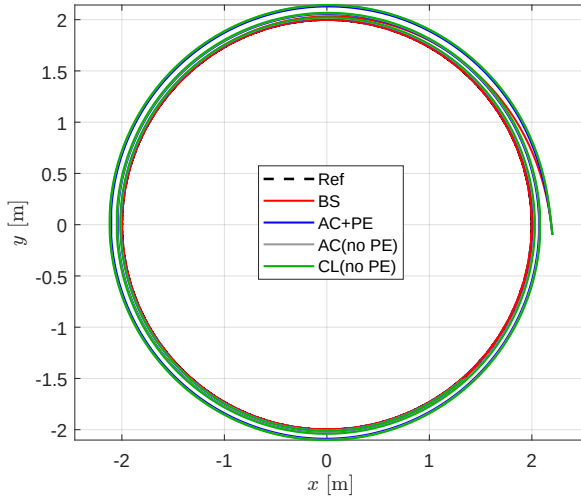


Figure 1: Quỹ đạo XY của bốn phương pháp trên đường tròn. Tất cả đều bám tốt, nhưng tốc độ hội tụ khác nhau.

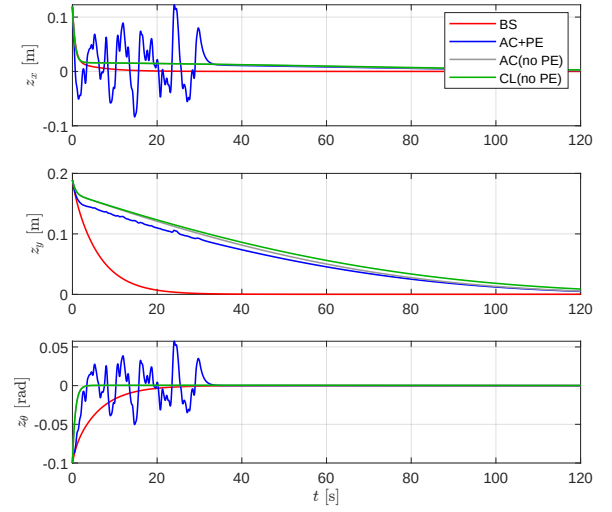


Figure 2: Sai lệch bám $z(t)$ theo thời gian trên đường tròn. CL hội tụ nhanh, gần tương đương AC+PE.

Nhận xét chi tiết:

1. **Backstepping** ($J_c = 1.35$): Tốt nhất về mọi chỉ số. Đây là kết quả mong đợi vì BS sử dụng trực tiếp mô hình, không cần quá trình học.
2. **CL** ($J_c = 8.31$) **gần tương đương AC+PE** ($J_c = 7.64$): Chênh lệch chỉ 8.8%, nhưng CL không sử dụng PE. Điều này chứng minh CL có thể thay thế PE mà hầu như không mất hiệu năng trên quỹ đạo thuận lợi (có kích thích tự nhiên).
3. **AC không PE** ($J_c = 12.48$) **kém nhất**: Trong ba phương pháp ADP, AC không PE cho kết quả tồi nhất. Điều này khẳng định: *PE là cần thiết cho Actor-Critic* — nếu bỏ PE, trọng số không hội tụ, dẫn đến hiệu năng kém.
4. **CL tốt hơn AC không PE 1.5×**: Chứng minh rằng history stack của CL thực sự cung cấp thông tin tương đương PE.

5.3 Phân tích chi tiết trên đường thẳng

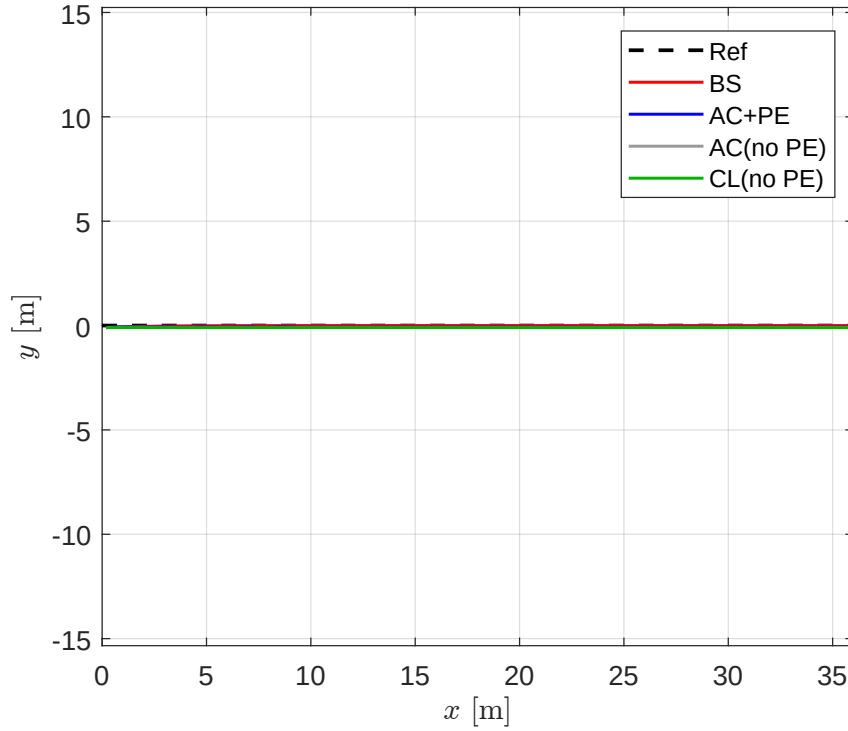


Figure 3: Quỹ đạo XY trên đường thẳng. CL (không PE) bám chính xác, trong khi AC+PE và AC không PE đều bị lệch đáng kể.

Đường thẳng là trường hợp “khó” nhất cho ADP vì $\omega_r = 0$, không có kích thích tự nhiên giữa các trạng thái. Kết quả cho thấy sự khác biệt rõ rệt:

- **AC + PE ($J_c = 72.71$):** Hiệu năng rất kém. Nguyên nhân: trên đường thẳng, drift $f = [0, v_r \sin z_\theta, 0]^T$ không có coupling ω_r . Tín hiệu PE thêm vào trở thành nhiễu thuần túy, không giúp khám phá đúng hướng mà còn phá hỏng quá trình bám quỹ đạo. $z_{\text{rms}} = 0.30$ m cho thấy robot vẫn đi lệch rất nhiều sau 120 s.
- **AC không PE ($J_c = 53.46$):** “Nghịch lý” là không có PE lại tốt hơn có PE ($53.46 < 72.71$), vì ít nhất không bị nhiễu PE phá. Tuy nhiên, trọng số không hội tụ (drift), $z_{\text{rms}} = 0.24$ m vẫn rất lớn.
- **CL ($J_c = 5.60$, $z_{\text{rms}} = 0.004$ m):** Vượt trội hoàn toàn. Hiệu năng gần bằng Backstepping ($J_c = 1.29$), và tốt hơn AC+PE **13 lần** ($72.71/5.60 \approx 13$). Sai lệch xác lập $z_{\text{rms}} = 0.004$ m (4 mm) — gần như hoàn hảo.

Giải thích chi tiết: Trên đường thẳng, $\omega_r = 0$ khiến drift f không có coupling — các trạng thái z_x, z_y, z_θ gần như độc lập. PE external thêm nhiễu *vô ích* (không giúp explore đúng hướng trong không gian tham số). Ngược lại, CL sử dụng dữ liệu lịch sử đã ghi từ giai đoạn đầu (khi sai lệch z còn lớn, σ đa dạng). Nhờ đó, CL tiếp tục học hiệu quả mà không cần bất kỳ kích thích mới nào.

5.4 Hội tụ trọng số

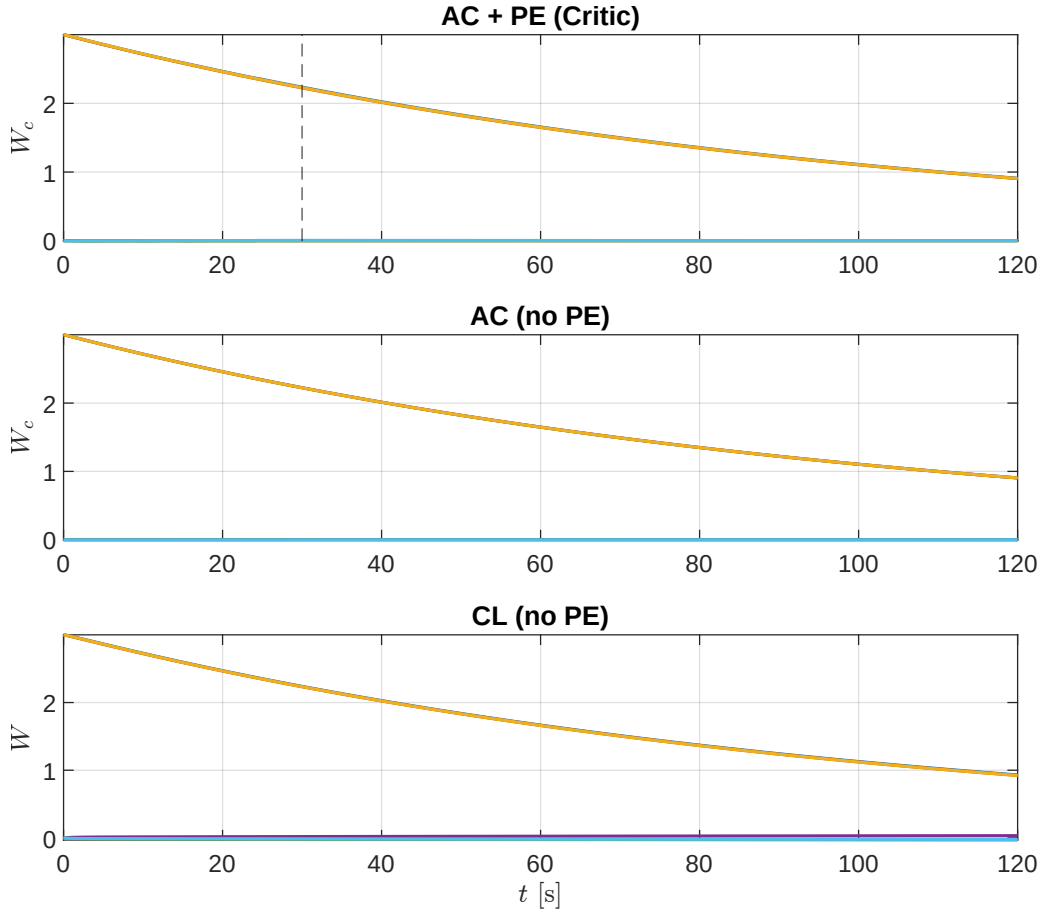


Figure 4: Quá trình hội tụ của 6 trọng số Critic trên đường tròn: AC+PE (trên), AC không PE (giữa), CL (dưới). CL hội tụ không cần PE, tốc độ tương đương AC+PE.

Sự hội tụ của trọng số là chỉ số quan trọng nhất để đánh giá chất lượng quá trình học. Khi trọng số hội tụ, chính sách điều khiển tiến gần nghiệm tối ưu của phương trình HJB.

Nhận xét chi tiết từ Hình 4:

1. **AC + PE:** Sáu trọng số hội tụ rõ ràng. Trong giai đoạn PE ($0 < t < 30$ s), trọng số dao động mạnh do nhiễu PE. Sau khi PE tắt ($t > 30$ s), trọng số ổn định và giữ giá trị xác lập. Đây là hành vi mong đợi khi PE hoạt động đúng.
2. **AC không PE:** Trọng số *không hội tụ* — vẫn drift chậm sau 120s, không đạt giá trị xác lập. Đây là bằng chứng trực quan rằng PE là cần thiết cho Actor-Critic: không có PE, gradient descent chỉ có một hướng $\bar{\sigma}$, không đủ thông tin để xác định W^* trong $\mathbb{R}^6$.
3. **CL:** Trọng số hội tụ *không cần PE*. Tốc độ hội tụ tương đương AC+PE, và đặc biệt không có dao động PE trong giai đoạn đầu. History stack cung cấp đủ thông tin (nhờ nhiều hướng σ_j khác nhau) để trọng số hội tụ về giá trị tối ưu.

5.5 Sai số Bellman

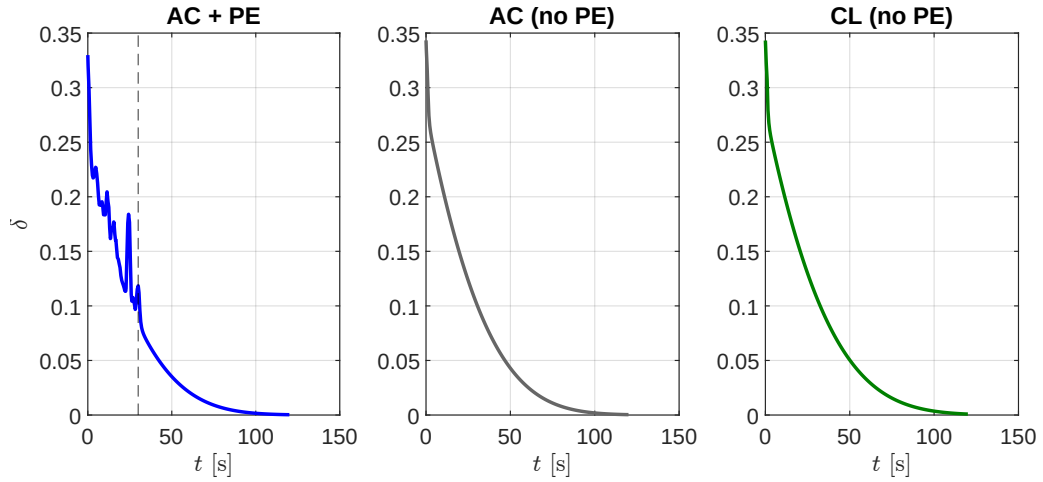


Figure 5: Sai số Bellman $\delta(t)$ (trung bình trượt — moving average) trên đường tròn. $\delta \rightarrow 0$ nghĩa là chính sách đang tiến về nghiệm tối ưu HJB.

Sai số Bellman $\delta(t)$ là thước đo trực tiếp “khoảng cách” giữa chính sách hiện tại và chính sách tối ưu. Khi $\delta \rightarrow 0$, chính sách điều khiển đã tiến gần nghiệm HJB.

Nhận xét từ Hình 5:

- **AC + PE:** $\delta \rightarrow 0$ sau khoảng 40 s. Trong 30 s đầu (giai đoạn PE), δ dao động lớn do nhiễu PE. Sau khi PE tắt, δ giảm nhanh về 0.
- **AC không PE:** δ giảm nhưng vẫn dao động, không tiến về 0 sau 120 s. Điều này khẳng định trọng số chưa đạt nghiệm HJB — chính sách điều khiển chưa tối ưu.
- **CL:** $\delta \rightarrow 0$ tương tự AC+PE, khẳng định CL hội tụ đến đúng nghiệm tối ưu. Đặc biệt, quá trình giảm của δ mượt hơn (không có dao động PE), cho thấy quá trình học ổn định hơn.

5.6 Chi phí tích lũy (Running cost)

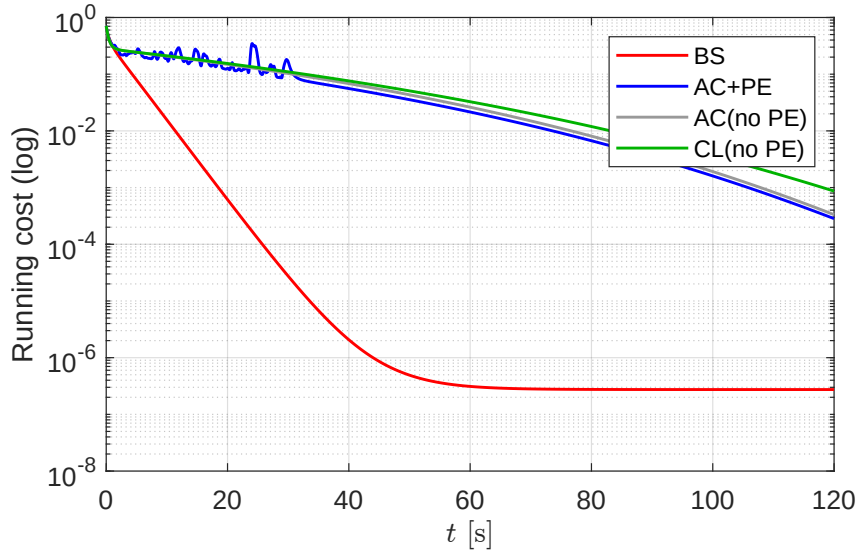


Figure 6: Chi phí tích lũy $J_c(t) = \int_0^t (z^\top Qz + u_o^\top Ru_o) d\tau$ trên đường tròn (thang logarit). BS thấp nhất, CL xếp ngay sau.

Hình 6 hiển thị chi phí tích lũy theo thời gian trên thang logarit. Chi phí tích lũy phản ánh tổng hợp cả sai lệch bám và năng lượng điều khiển — một thước đo “toàn diện” về chất lượng điều khiển.

- Backstepping có chi phí thấp nhất (model-based, tối ưu nhất).
- CL xếp ngay sau BS, vượt trội cả AC+PE và AC không PE.
- AC không PE có chi phí tăng liên tục (trọng số không hội tụ, sai lệch không giảm về 0).

6 Thảo luận

6.1 CL và AC+PE: khi nào CL tốt hơn?

Bảng 3 tổng hợp so sánh giữa CL và AC+PE theo nhiều tiêu chí.

Table 3: So sánh toàn diện CL và AC+PE

Tiêu chí	AC+PE	CL
$\omega_r \neq 0$ (đường tròn)	Tốt	Tương đương (chênh 8.8%)
$\omega_r = 0$ (đường thẳng)	Kém (PE phản tác dụng)	Tốt hơn nhiều (13×)
Yêu cầu PE signal	Có	Không
Số mạng nơ-ron	2 (Actor + Critic)	1 (Critic-only)
Số siêu tham số	$\alpha_c, \alpha_{a1}, \alpha_{a2}, \kappa_c + \text{PE}$	α, α_h, κ
Robust với quỹ đạo	Không	Có

Kết luận: CL ưu việt hơn AC+PE trên hầu hết các tiêu chí. Đặc biệt, CL không phụ thuộc vào đặc điểm của quỹ đạo tham chiếu — một ưu điểm rất quan trọng trong ứng dụng thực tế, khi quỹ đạo có thể thay đổi liên tục.

Trên đường tròn, AC+PE nhỉnh hơn một chút ($J_c = 7.64$ so với 8.31) vì PE cung cấp thêm kích thích có lợi. Tuy nhiên, sự khác biệt này là nhỏ và được bù đắp hoàn toàn bởi ưu thế của CL trên đường thẳng.

6.2 Hạn chế còn lại và hướng phát triển

Mặc dù CL giải quyết được vấn đề PE, vẫn còn ba hạn chế chưa được giải quyết:

1. **Vấn đề warm start:** CL không giải quyết bài toán chính sách ban đầu hợp lệ (admissible initial policy). $W_0 = 0$ vẫn gây mất ổn định. Trong thực tế, cần một bộ điều khiển nền (như P hoặc backstepping) để khởi tạo W_0 hợp lệ.
2. **Chỉ xét kinematic (vòng ngoài):** Mô phỏng hiện tại giả định lệnh vận tốc được thực hiện chính xác bởi động cơ. Trong thực tế, cần thêm vòng trong động lực học để xử lý quán tính, ma sát, và nhiễu loạn.
3. **Chưa có ràng buộc thời gian hội tụ (fixed-time):** Hội tụ hiện tại là asymptotic (tiệm cận) — không có cận trên cho thời gian hội tụ. Trong ứng dụng thực tế, cần đảm bảo robot bám đúng quỹ đạo trong một khoảng thời gian định trước (fixed-time convergence).

Hướng đi cho luận văn: Kết hợp Concurrent Learning với:

- **Fixed-time robust terms** (Wang et al., 2025): đảm bảo hội tụ trong thời gian có định, không phụ thuộc điều kiện ban đầu.
- **Kiến trúc dual-loop** (kinematic + dynamic): vòng ngoài điều khiển quỹ đạo, vòng trong điều khiển mô-men động cơ.
- **Xử lý nhiễu loạn và bất định mô hình:** sử dụng các thành phần robust như sliding mode.

Đây sẽ là nội dung chính của luận văn: *bộ điều khiển ADP fixed-time dual-loop với Concurrent Learning cho robot di động bánh xe*.

7 Kết luận

Báo cáo Seminar 2 đã trình bày việc thiết kế, cài đặt và đánh giá bộ điều khiển Critic-only ADP kết hợp Concurrent Learning cho bài toán bám quỹ đạo robot di động bánh xe. Bốn kết luận chính:

1. **Critic-only + CL thay thế thành công Actor-Critic + PE:** Giảm từ 2 mạng xuống 1 mạng, loại bỏ hoàn toàn tín hiệu PE. Hiệu năng trên đường tròn tương đương (chênh lệch dưới 9%), chứng minh CL cung cấp đủ thông tin thay thế PE.

2. **CL vượt trội trên đường thẳng:** $J_c = 5.60$ so với 72.71 của AC+PE — tốt hơn 13 lần. CL không bị ảnh hưởng bởi đặc điểm quỹ đạo, trong khi AC+PE thất bại khi quỹ đạo không có kích thích tự nhiên.
3. **History stack thay thế PE hiệu quả:** Với 200 điểm dữ liệu, ghi mỗi 50 ms, điều kiện rank tự động thỏa mãn sau khoảng 3 giây vận hành. Cài đặt vectorized trong MATLAB đảm bảo tốc độ tính toán nhanh (khoảng $100\times$ so với vòng lặp).
4. **Nền tảng cho luận văn:** Cấu trúc Critic-only với Concurrent Learning sẽ được mở rộng trong giai đoạn tiếp theo với các thành phần fixed-time robust và kiến trúc dual-loop (kinematic + dynamic), hướng tới bộ điều khiển hoàn chỉnh cho robot di động bánh xe trong điều kiện thực tế.

References

- [1] Y. Wang, Z. Li, et al., “Adaptive dynamic programming-based fixed-time optimal control for wheeled mobile robot,” *IEEE Robotics and Automation Letters*, vol. 10, no. 1, pp. 176–183, Jan. 2025.
- [2] K. G. Vamvoudakis and F. L. Lewis, “Online actor-critic algorithm to solve the continuous-time infinite horizon optimal control problem,” *Automatica*, vol. 46, no. 5, pp. 878–888, 2010.
- [3] G. Chowdhary and E. Johnson, “Concurrent learning for convergence in adaptive control without persistency of excitation,” in *Proc. IEEE Conf. Decision and Control*, 2010, pp. 3674–3679.
- [4] Y. Kanayama, Y. Kimura, F. Miyazaki, and T. Noguchi, “A stable tracking control method for an autonomous mobile robot,” in *Proc. IEEE Int. Conf. Robotics and Automation*, 1990, pp. 384–389.
- [5] R. Fierro and F. L. Lewis, “Control of a nonholonomic mobile robot: backstepping kinematics into dynamics,” *J. Robotic Systems*, vol. 14, no. 3, pp. 149–163, 1997.